

LABORATORIO 11

El disco lleno

Guía para leer mientras trabajas el cuaderno. Cada celda trae la pregunta que responde, la idea en el almacén, la sentencia explicada parte por parte, lo que sale en pantalla y cómo leerlo. Las salidas son las de la solución ejecutada del repositorio. Los tamaños van en MB de un millón de bytes.

Introducción

1 · El tema

En este laboratorio vas a medir con números cuánto espacio ocupa la historia de una tabla, y qué pasa en el disco si nadie la ordena. La pregunta es la que siempre hace el que paga el almacenamiento. Si la tabla tiene las mismas treinta mil filas que ayer, ¿por qué ocupa varias veces más?

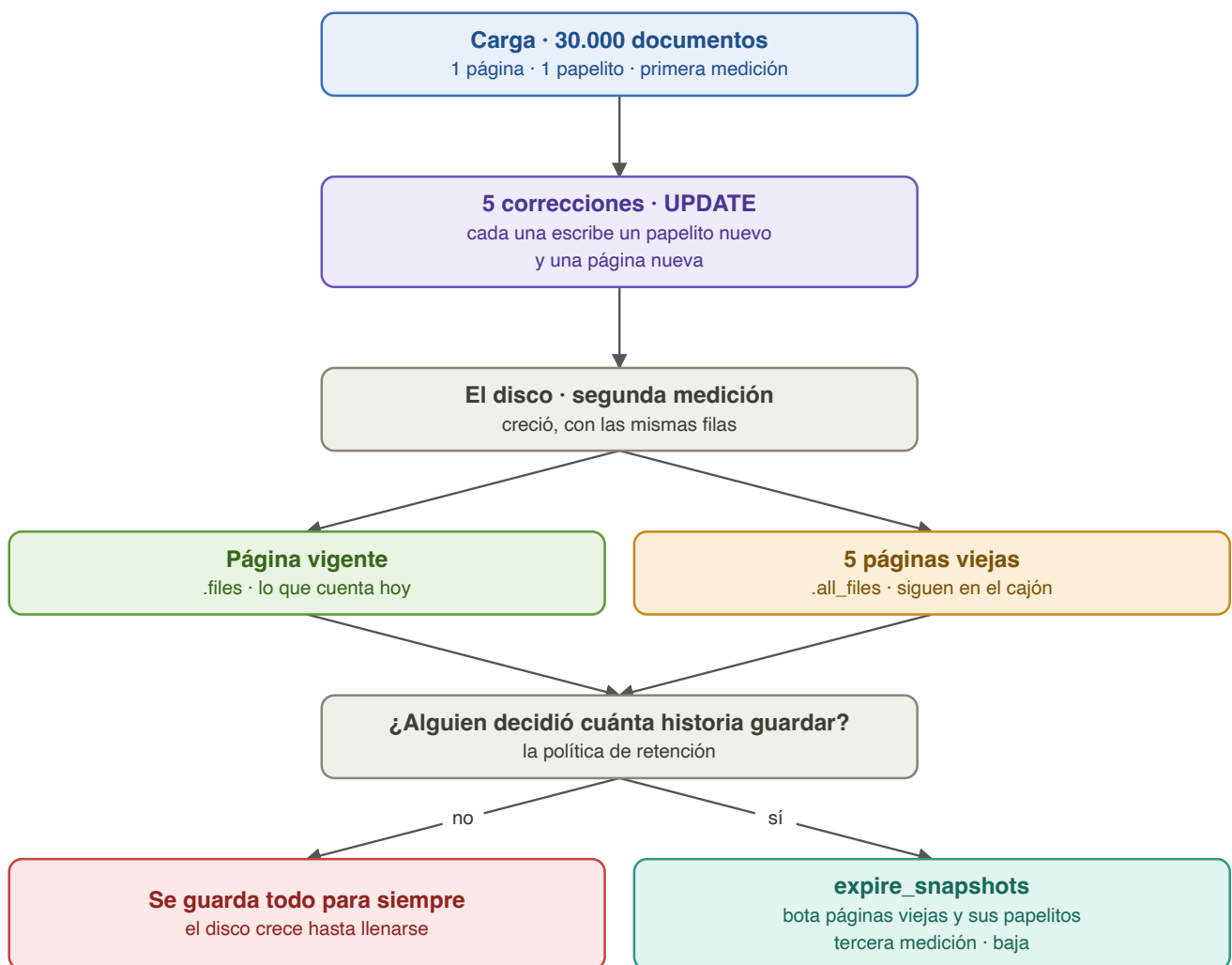
2 · El problema, en el almacén

En el almacén, cada vez que alguien corrige algo en la libreta, no borra la página anterior. Escribe una página nueva con la corrección y deja la vieja en el cajón. Gracias a eso se puede saber cómo estaba todo el martes pasado. Pero el cajón se va llenando. Nadie bota las páginas viejas hasta que alguien lo ordena, y un día el cajón no cierra.

3 · El problema en términos técnicos

En Iceberg, un `UPDATE` no modifica los archivos Parquet que ya existen. Escribe archivos nuevos con las filas corregidas y cuelga una página nueva que apunta a ellos. Los archivos viejos quedan en el disco, porque las páginas viejas todavía los nombran. La tabla no cambia en filas, pero en disco crece con cada corrección. Y si un proceso se corta a la mitad, puede dejar archivos que ninguna página nombra, los huérfanos.

4 · El diagrama



Carga · 30.000 documentos (azul). Partes con un año de documentos, treinta mil filas, en una página y un papelito. Ahí tomas la primera medición del disco.

5 correcciones · UPDATE (morado). Cinco `UPDATE` seguidos, uno por mes, de enero a mayo. Ninguno agrega ni quita filas, solo cambian el estado de algunos documentos. Pero cada uno escribe un papelito nuevo con las filas corregidas y cuelga una página nueva.

El disco · segunda medición (gris). La segunda medición muestra que el disco creció, aunque la tabla tiene las mismas treinta mil filas.

Página vigente (verde). Es lo que cuenta hoy, lo que ve cualquier consulta. Se mira con la vista `.files`.

5 páginas viejas (amarillo). Son las páginas de antes de cada corrección, con sus papelitos. Siguen en el disco. Se ven con la vista `.all_files`, que muestra todos los archivos, los vigentes y los viejos. La diferencia entre las dos vistas es el espacio que se fue.

¿Alguien decidió cuánta historia guardar? (gris). Aquí se decide todo. Iceberg no bota nada por su cuenta. Alguien tiene que decidir cuánta historia se conserva, y esa decisión se llama política de retención.

Se guarda todo para siempre (rojo). Si nadie decide, se guarda todo. El disco crece con cada corrección hasta que se llena.

expire_snapshots (verde agua). Si alguien decide, se bota la historia que ya no se necesita.

expire_snapshots elimina las páginas viejas y los papelitos que solo ellas nombraban, y en la tercera medición el disco baja.

5 · La solución

En el almacén, el dueño decide cuánto tiempo se guardan las páginas viejas. Por ejemplo, un año, porque el contador puede pedir revisar hasta un año atrás. Una vez al mes alguien abre el cajón y bota las páginas más viejas que eso, y lo que se bota no se recupera.

En Iceberg es igual. La historia no es un defecto, es lo que permite consultar la tabla como estaba en una fecha pasada. Pero tiene un costo en disco, y se controla con una **política de retención** y dos procedimientos.

expire_snapshots bota las páginas más viejas que una fecha, conservando las últimas que uno diga.

remove_orphan_files bota los archivos que ninguna página nombra. La política se decide con dos preguntas. Cuánto hacia atrás hay que poder consultar, que fija el mínimo, y cuánto disco hay, que fija el máximo.

6 · Los pasos

- **Paso 0.** Creas una tabla con un año de documentos, treinta mil filas.
- **Paso 1.** Primera medición, preguntándole a la libreta y al disco. No dan lo mismo.
- **Paso 2.** Cinco correcciones. Compruebas que siguen siendo treinta mil filas y mides de nuevo el disco.
- **Paso 3.** Buscas dónde se fue el espacio, con las seis páginas y la diferencia entre **.files** y **.all_files**.
- **Paso 4.** Botas las páginas viejas con **expire_snapshots**, mides por tercera vez y compruebas que los datos no se tocaron.
- **Paso 5.** Buscas archivos huérfanos con **remove_orphan_files**.
- **Cierre.** Los tres números del disco y la política de retención.

Para repetir el laboratorio desde cero, borra su tabla con el comando de abajo, desde la carpeta del repositorio. La celda 0.2 también la borra.

```
bash
bin/reiniciar-lab.sh 11
```

Paso 0 · Un año de documentos

Celda 0.1 · Ponerte en tu espacio

La pregunta. ¿En qué espacio vas a crear la tabla?

Por qué ahora. La tabla de hoy es tuya. Vas a hacer tus propias correcciones y medir tu propio disco, así que lo primero es ponerte en tu espacio.

En el almacén. Es elegir tu estante en la bodega. La libreta de hoy va a quedar en tu estante, y lo que midas en el disco va a ser solo lo tuyo.

La sentencia, parte por parte.

- `%%sql` es la primera línea de toda celda SQL del cuaderno. Le dice al cuaderno que lo que sigue es SQL y no Python.
- `-- Celda 0.1` es un comentario. Lo que va después de dos guiones lo ignora el motor.
- `USE` cambia el espacio con el que trabajas.
- `mi_espacio` es tu espacio. En el disco tiene su carpeta, `/warehouse/iceberg/mi_espacio.db/`, y ahí va a vivir la tabla que vas a medir.

```
sql
%%sql
-- Celda 0.1
USE mi_espacio
```

Lo que sale en pantalla.

```
text
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/09/25 18:09:20 WARN NativeCodeLoader: Unable to load native-hadoop library for your
platform... using builtin-java classes where applicable
Listo. La sentencia se ejecutó.
```

Cómo se lee. La última línea dice que la sentencia se ejecutó. Las tres de arriba son avisos de Spark al arrancar, y salen solo en la primera celda.

- `Setting default log level to "WARN"` dice que Spark va a mostrar solo los avisos y los errores.
- `To adjust logging level use sc.setLogLevel(newLevel)` explica cómo cambiar eso. `sc` es el contexto de Spark y `setLogLevel` la función que cambia el nivel. No hace falta tocarlo. SparkR es la versión de Spark para el lenguaje R, que aquí no se usa.
- `WARN NativeCodeLoader: Unable to load native-hadoop library` dice que Spark no encontró una biblioteca de Hadoop escrita para este sistema operativo y que usa la versión en Java. Funciona igual. `26/09/25 18:09:20` es la fecha y la hora del aviso, año 2026, mes 09, día 25, en hora UTC.
- `Listo. La sentencia se ejecutó.` es el aviso del cuaderno cuando una sentencia no devuelve filas.

Celda 0.2 · Partir de cero

La pregunta. ¿Y si la tabla ya existe?

Por qué ahora. Si quedó una tabla de una corrida anterior, con páginas y papelitos viejos, las mediciones del disco salen infladas.

En el almacén. Es descolgar del clavo cualquier libreta vieja con este nombre, para colgar una nueva en blanco.

La sentencia, parte por parte.

- `DROP TABLE` borra la tabla del catálogo.
- `IF EXISTS` la borra solo si existe. Si no existe, no reclama.
- `documentos_lab11` es el nombre de la tabla, sin el espacio adelante porque ya estás en `mi_espacio`.

```
sql
%%sql
-- Celda 0.2
DROP TABLE IF EXISTS documentos_lab11
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. El aviso es el mismo exista o no la tabla.

Celda 0.3 · Crear la tabla con un año de documentos

La pregunta. ¿Con qué datos vas a medir?

Por qué ahora. Hace falta una tabla de un tamaño que se note en el disco. Un año de documentos tributarios, treinta mil filas, alcanza.

En el almacén. Es copiar a una libreta nueva todas las anotaciones de un año, de una vez. Queda una página y un papelito.

La sentencia, parte por parte.

- `CREATE TABLE documentos_lab11` crea la tabla en `mi_espacio`.
- `USING iceberg` la deja en formato Iceberg.
- `AS SELECT * FROM curso.dte_2024` la crea y la llena, en la misma sentencia, con todo lo que tiene la tabla `curso.dte_2024`. `curso` es el espacio de las tablas de referencia del ambiente y `dte_2024` tiene los documentos tributarios de 2024. El asterisco trae todas las columnas.

```
sql
%%sql
-- Celda 0.3
CREATE TABLE documentos_lab11 USING iceberg AS
SELECT * FROM curso.dte_2024
```

Lo que sale en pantalla.

```
text
```

Listo. La sentencia se ejecutó.

Cómo se lee. La tabla quedó creada y cargada, con una sola página.

Paso 1 · Medir el disco

Celda 1.1 · Preguntarle a la libreta

La pregunta. ¿Cuántos papelitos cuentan hoy y cuánto pesan?

Por qué ahora. Es la primera de dos formas de medir. Esta le pregunta a la libreta, que sabe qué papelitos forman la página vigente.

En el almacén. Es abrir la libreta en la página de hoy y sumar el peso de los papelitos que nombra.

La sentencia, parte por parte.

- `count(*)` cuenta las filas de la vista, y cada fila es un papelito. `AS archivos_vigentes` le pone nombre a la columna.
- `sum(file_size_in_bytes)` suma el tamaño en bytes de cada papelito. `AS bytes_vigentes` es su nombre.
- `FROM mi_espacio.documentos_lab11.files` lee la vista de sistema `.files`, que tiene una fila por cada papelito de la página vigente. El nombre va con tres partes, el espacio, la tabla y la vista.

```
sql
%%sql
-- Celda 1.1
SELECT count(*)           AS archivos_vigentes,
       sum(file_size_in_bytes) AS bytes_vigentes
FROM mi_espacio.documentos_lab11.files
```

Lo que sale en pantalla.

```
text
archivos_vigentes  bytes_vigentes
1                  776514
1 fila.
```

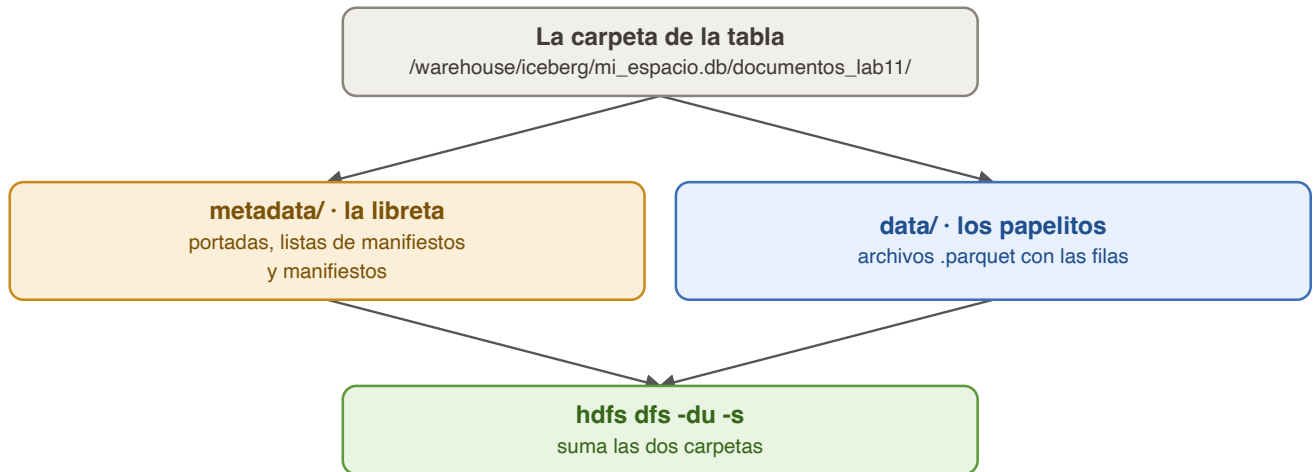
Cómo se lee. Un papelito de 0,78 MB (776.514 bytes). Ahí están los treinta mil documentos. Al pie, `1 fila.` es el conteo de filas del resultado.

Celda 1.2 · Preguntarle al disco

La pregunta. ¿Cuánto ocupa la carpeta de la tabla en el disco?

Por qué ahora. Es la segunda forma de medir. El disco no sabe nada de libretas y solo ve archivos. Es lo que mira el que paga el almacenamiento. La celda viene escrita en el cuaderno.

En el almacén. Es pesar el cajón entero, con todo lo que tiene adentro, sin leer nada.



La carpeta de la tabla (gris). Cada tabla es una carpeta en HDFS, dentro de la carpeta de su espacio.

metadata/ · la libreta (amarillo). Aquí están la portada, que es el archivo `metadata.json`, las listas de manifiestos de cada página y los manifiestos. No tienen ni una fila de datos, pero pesan algo.

data/ · los papelitos (azul). Aquí están los archivos Parquet con las filas. Es lo que suma la celda 1.1.

hdfs dfs -du -s (verde). La orden mide las dos carpetas juntas.

La sentencia, parte por parte.

- `# Celda 1.2` es un comentario de Python, que el cuaderno no ejecuta.
- `!` manda la línea al sistema operativo y no a Spark. Por eso la celda no lleva `%%sql`.
- `hdfs dfs` es el cliente de línea de comandos de HDFS.
- `-du` quiere decir uso de disco.
- `-s` pide un solo total para la carpeta, en vez de una línea por cada archivo.
- `/warehouse/iceberg/mi_espacio.db/documentos_lab11` es la carpeta de la tabla.

```
bash
# Celda 1.2
!hdfs dfs -du -s /warehouse/iceberg/mi_espacio.db/documentos_lab11
```

Lo que sale en pantalla.

```
text
790816 2372448 /warehouse/iceberg/mi_espacio.db/documentos_lab11
```

Cómo se lee. Tres columnas. El primer número es lo que pesan los archivos, 0,79 MB (790.816 bytes). El segundo es lo que ocupan contando las copias de respaldo que HDFS pide, 2,37 MB (2.372.448 bytes). La tercera columna es la carpeta medida.

El primer número es un poco mayor que el de la celda 1.1, 14.302 bytes más, porque el disco cuenta también la carpeta `metadata`. El segundo es exactamente el triple del primero, porque HDFS guarda cada archivo con factor de replicación 3. En las preguntas frecuentes está explicado.

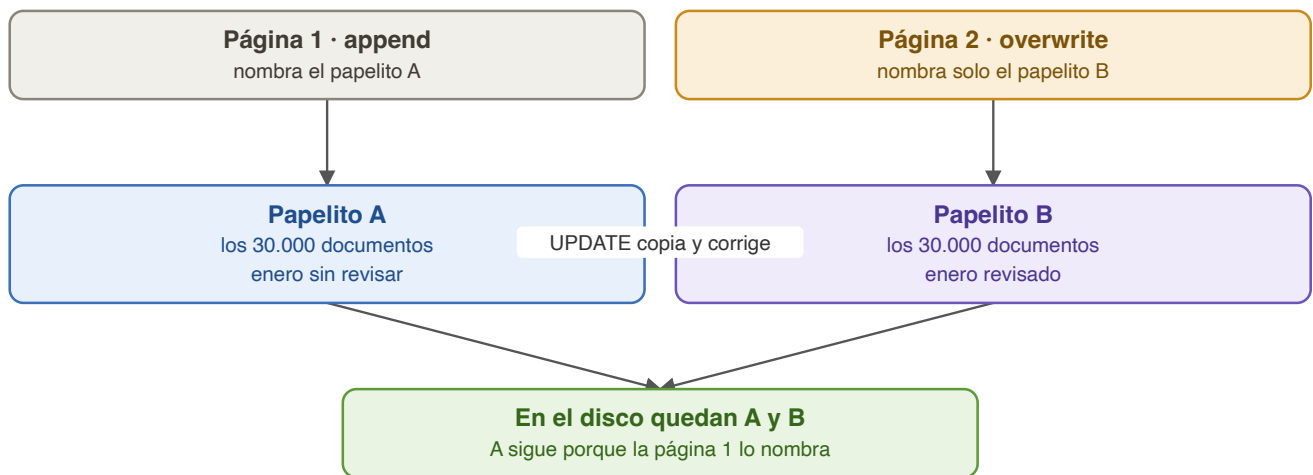
Paso 2 · Cinco correcciones

Celda 2.1 · Primera corrección, enero

La pregunta. ¿Qué le pasa al disco cuando corriges datos sin agregar ni quitar filas?

Por qué ahora. Vas a hacer cinco correcciones seguidas, una por mes, de enero a mayo. Ninguna agrega ni quita filas. Si la tabla sigue con treinta mil filas, uno esperaría que el disco quede igual.

En el almacén. Para corregir un renglón, el almacenero no borra con goma. Copia el papelito completo en uno nuevo, con la corrección, y cuelga una página que dice que ahora vale el nuevo. El papelito viejo queda en el cajón, porque la página anterior todavía lo nombra.



Página 1 · append (gris). La página de la carga nombra un solo papelito.

Papelito A (azul). Tiene los treinta mil documentos, con enero sin revisar.

Papelito B (morado). Iceberg nunca modifica un archivo Parquet que ya existe. Lee el papelito A, cambia las filas de enero y escribe un papelito nuevo. Como toda la tabla está en un solo papelito, el nuevo lleva los treinta mil documentos completos. Esta forma de corregir se llama copiar al escribir.

Página 2 · overwrite (amarillo). La corrección cuelga una página nueva que nombra solo el papelito B. Esa pasa a ser la vigente.

En el disco quedan A y B (verde). El papelito A no se borra, porque la página 1 todavía lo nombra y alguien podría pedir la tabla como estaba ahí.

La sentencia, parte por parte.

- `UPDATE documentos_lab11` corrige filas que ya existen en la tabla.
- `SET estado_sii = 'REVISADO'` cambia la columna `estado_sii`, el estado del documento, al valor `REVISADO`.
- `WHERE tipo_dte = 33` se queda con las facturas electrónicas, que son el tipo de documento 33.
- `AND fecha_emision BETWEEN DATE '2024-01-01' AND DATE '2024-01-31'` se queda con las emitidas en enero de 2024. `BETWEEN` incluye las dos fechas del borde y `DATE` dice que el texto es una fecha.

```
sql
%%sql
-- Celda 2.1
UPDATE documentos_lab11 SET estado_sii = 'REVISADO'
WHERE tipo_dte = 33 AND fecha_emision BETWEEN DATE '2024-01-01' AND DATE '2024-01-31'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. En la pantalla no se ve nada más, pero en el disco ya hay dos papelitos grandes.

Celda 2.2 · Segunda corrección, febrero

La pregunta. ¿Qué deja la segunda corrección?

Por qué ahora. Cada corrección suma un papelito y una página. Febrero es la segunda.

En el almacén. El almacenero vuelve a copiar el papelito completo, ahora con febrero corregido, y cuelga otra página.

La sentencia, parte por parte.

- `UPDATE documentos_lab11` corrige filas de la tabla.
- `SET estado_sii = 'REVISADO'` pone el estado `REVISADO`.
- `WHERE tipo_dte = 33` se queda con las facturas electrónicas.
- `AND fecha_emision BETWEEN DATE '2024-02-01' AND DATE '2024-02-29'` se queda con febrero de 2024, que tuvo 29 días.

```
sql
%%sql
-- Celda 2.2
UPDATE documentos_lab11 SET estado_sii = 'REVISADO'
WHERE tipo_dte = 33 AND fecha_emision BETWEEN DATE '2024-02-01' AND DATE '2024-02-29'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. Ya van tres papelitos en el disco y tres páginas.

Celda 2.3 · Tercera corrección, marzo

La pregunta. ¿Y la tercera?

Por qué ahora. Sigue la serie, con marzo.

En el almacén. Otra copia completa del papelito, con marzo corregido, y otra página.

La sentencia, parte por parte.

- `UPDATE documentos_lab11` corrige filas de la tabla.
- `SET estado_sii = 'REVISADO'` pone el estado `REVISADO`.
- `WHERE tipo_dte = 33` se queda con las facturas electrónicas.
- `AND fecha_emision BETWEEN DATE '2024-03-01' AND DATE '2024-03-31'` se queda con marzo de 2024.

```
sql
%%sql
-- Celda 2.3
UPDATE documentos_lab11 SET estado_sii = 'REVISADO'
WHERE tipo_dte = 33 AND fecha_emision BETWEEN DATE '2024-03-01' AND DATE '2024-03-31'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. Cuatro papelitos y cuatro páginas.

Celda 2.4 · Cuarta corrección, abril

La pregunta. ¿Y la cuarta?

Por qué ahora. Sigue la serie, con abril.

En el almacén. Otra copia completa del papelito, con abril corregido, y otra página.

La sentencia, parte por parte.

- `UPDATE documentos_lab11` corrige filas de la tabla.
- `SET estado_sii = 'REVISADO'` pone el estado `REVISADO`.
- `WHERE tipo_dte = 33` se queda con las facturas electrónicas.
- `AND fecha_emision BETWEEN DATE '2024-04-01' AND DATE '2024-04-30'` se queda con abril de 2024.

```
sql
%%sql
-- Celda 2.4
UPDATE documentos_lab11 SET estado_sii = 'REVISADO'
WHERE tipo_dte = 33 AND fecha_emision BETWEEN DATE '2024-04-01' AND DATE '2024-04-30'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. Cinco papelitos y cinco páginas.

Celda 2.5 · Quinta corrección, mayo

La pregunta. ¿Y la última?

Por qué ahora. Cierra la serie, con mayo.

En el almacén. La quinta copia completa del papelito, con mayo corregido, y la sexta página.

La sentencia, parte por parte.

- `UPDATE documentos_lab11` corrige filas de la tabla.
- `SET estado_sii = 'REVISADO'` pone el estado `REVISADO`.
- `WHERE tipo_dte = 33` se queda con las facturas electrónicas.
- `AND fecha_emision BETWEEN DATE '2024-05-01' AND DATE '2024-05-31'` se queda con mayo de 2024.

```
sql
%%sql
-- Celda 2.5
UPDATE documentos_lab11 SET estado_sii = 'REVISADO'
WHERE tipo_dte = 33 AND fecha_emision BETWEEN DATE '2024-05-01' AND DATE '2024-05-31'
```

Lo que sale en pantalla.

```
text
Listo. La sentencia se ejecutó.
```

Cómo se lee. Se ejecutó. Seis papelitos en el disco y seis páginas en la libreta.

Celda 2.6 · Contar las filas

La pregunta. ¿La tabla sigue teniendo las mismas filas?

Por qué ahora. Antes de medir el disco hay que asegurarse de que las correcciones no agregaron ni quitaron nada. Si las filas son las mismas, cualquier crecimiento del disco es historia y no datos.

En el almacén. Es contar los renglones de la página de hoy.

La sentencia, parte por parte.

- `count(*)` cuenta todas las filas.
- `AS documentos` le pone nombre a la columna.
- `FROM documentos_lab11` es la tabla de hoy.

```
sql
%%sql
-- Celda 2.6
SELECT count(*) AS documentos FROM documentos_lab11
```

Lo que sale en pantalla.

```
text
documentos
30000
1 fila.
```

Cómo se lee. Los mismos 30000 documentos del principio. Ninguna corrección agregó ni quitó filas.

Celda 2.7 · Medir el disco otra vez

La pregunta. ¿Cuánto ocupa ahora la carpeta de la tabla?

Por qué ahora. Es la segunda de las tres mediciones. La celda es igual a la 1.2 y viene escrita en el cuaderno.

En el almacén. Es volver a pesar el cajón, después de las cinco correcciones.

La sentencia, parte por parte.

- `!` manda la línea al sistema operativo.
- `hdfs dfs -du -s` pide el uso de disco total de una carpeta.
- `/warehouse/iceberg/mi_espacio.db/documentos_lab11` es la carpeta de la tabla.

```
bash
# Celda 2.7
!hdfs dfs -du -s /warehouse/iceberg/mi_espacio.db/documentos_lab11
```

Lo que sale en pantalla.

```
text
4805132 14415396 /warehouse/iceberg/mi_espacio.db/documentos_lab11
```

Cómo se lee. El disco pasó de 0,79 MB a 4,81 MB (4.805.132 bytes), con las mismas filas. Es unas seis veces más. El segundo número, 14,42 MB (14.415.396 bytes), sigue siendo el triple del primero.

Paso 3 · Dónde se fue el espacio

Celda 3.1 · Las páginas de la libreta

La pregunta. ¿Qué páginas hay en la libreta después de las cinco correcciones?

Por qué ahora. Para saber dónde se fue el espacio, primero se mira la historia. Si cada corrección dejó una página, tiene que haber seis.

En el almacén. Es abrir la libreta en la lista de páginas y contarlas. Cada página tiene su papelito guardado en el cajón.

La sentencia, parte por parte.

- `SELECT snapshot_id, committed_at, operation` pide el número de cada página, la hora en que se colgó, en UTC, y qué se hizo en ella.
- `FROM mi_espacio.documentos_lab11.snapshots` lee la vista de sistema con la lista de páginas.
- `ORDER BY committed_at` ordena de la más vieja a la más nueva.

```
sql
%%sql
-- Celda 3.1
SELECT snapshot_id, committed_at, operation
FROM mi_espacio.documentos_lab11.snapshots
ORDER BY committed_at
```

Lo que sale en pantalla.

```
text
snapshot_id      committed_at      operation
3555416021291169985  2026-09-25 18:09:27.985  append
8101134315995455764  2026-09-25 18:09:31.986  overwrite
1029929284241993774  2026-09-25 18:09:33.432  overwrite
7443930311595880363  2026-09-25 18:09:34.236  overwrite
4372640588754442152  2026-09-25 18:09:35.021  overwrite
1180009671557740938  2026-09-25 18:09:36.187  overwrite
6 filas.
```

Cómo se lee. Seis páginas, la carga y las cinco correcciones.

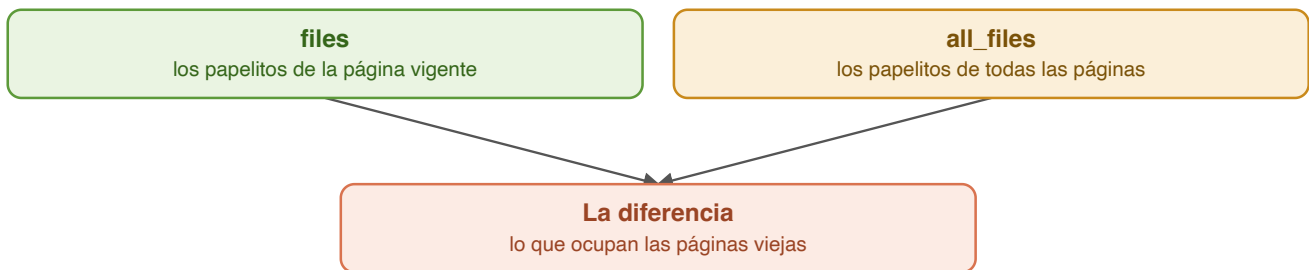
- La primera, `3555416021291169985`, es `append`, que significa que se agregaron filas. Es la carga de la celda 0.3, de las 18:09:27.985 UTC, las 15:09 en Chile.
- Las otras cinco son `overwrite`, que significa que se reemplazaron papelitos. Son los cinco `UPDATE`, entre las 18:09:31.986 y las 18:09:36.187 UTC. La última, `1180009671557740938`, es la vigente.

Celda 3.2 · Lo que cuenta hoy contra lo que hay

La pregunta. ¿Cuánto del disco es la tabla de hoy y cuánto es historia?

Por qué ahora. Es la comparación que lo explica todo. La vista `.files` son los papelitos que cuentan hoy y `.all_files` son todos los que nombra alguna página, incluidas las viejas.

En el almacén. Es pesar por separado los papelitos de la página de hoy y todos los papelitos que hay en el cajón.



files (verde). Es la vista de los papelitos que forman la página vigente. Es lo que lee cualquier consulta de hoy.

all_files (amarillo). Es la vista de los papelitos que nombra cualquiera de las páginas, la vigente y las viejas.

La diferencia (coral). Lo que pesa `.all_files` y no pesa `.files` es lo que ocupa la historia.

La sentencia, parte por parte. Son cuatro consultas chicas dentro de una, cada una entre paréntesis, que devuelven un número cada una.

- `(SELECT count(*) FROM mi_espacio.documentos_lab11.files) AS archivos_vigentes` cuenta los papelitos de la página vigente.
- `(SELECT sum(file_size_in_bytes) FROM mi_espacio.documentos_lab11.files) AS bytes_vigentes` suma lo que pesan.
- `(SELECT count(*) FROM mi_espacio.documentos_lab11.all_files) AS archivos_todos` cuenta los papelitos de todas las páginas.
- `(SELECT sum(file_size_in_bytes) FROM mi_espacio.documentos_lab11.all_files) AS bytes_todos` suma lo que pesan todos.
- El `SELECT` de afuera no tiene `FROM`, porque solo junta los cuatro números en una fila.

```
sql
%%sql
-- Celda 3.2
SELECT
  (SELECT count(*)
   FROM mi_espacio.documentos_lab11.files) AS
archivos_vigentes,
  (SELECT sum(file_size_in_bytes) FROM mi_espacio.documentos_lab11.files) AS
bytes_vigentes,
  (SELECT count(*)
   FROM mi_espacio.documentos_lab11.all_files) AS
archivos_todos,
  (SELECT sum(file_size_in_bytes) FROM mi_espacio.documentos_lab11.all_files) AS bytes_todos
```

Lo que sale en pantalla.

```
text
archivos_vigentes  bytes_vigentes  archivos_todos  bytes_todos
1                  778984         6               4668576
1 fila.
```

Cómo se lee. Hoy cuenta un papelito de 0,78 MB (778.984 bytes), pero hay seis papelitos que pesan 4,67 MB (4.668.576 bytes). Casi todo el espacio es historia.

El papelito vigente pesa un poco distinto que el de la celda 1.1, porque es otro archivo, el que escribió la última corrección, y la compresión depende de su contenido. Los seis son la carga y las cinco copias, cada una con los

treinta mil documentos.

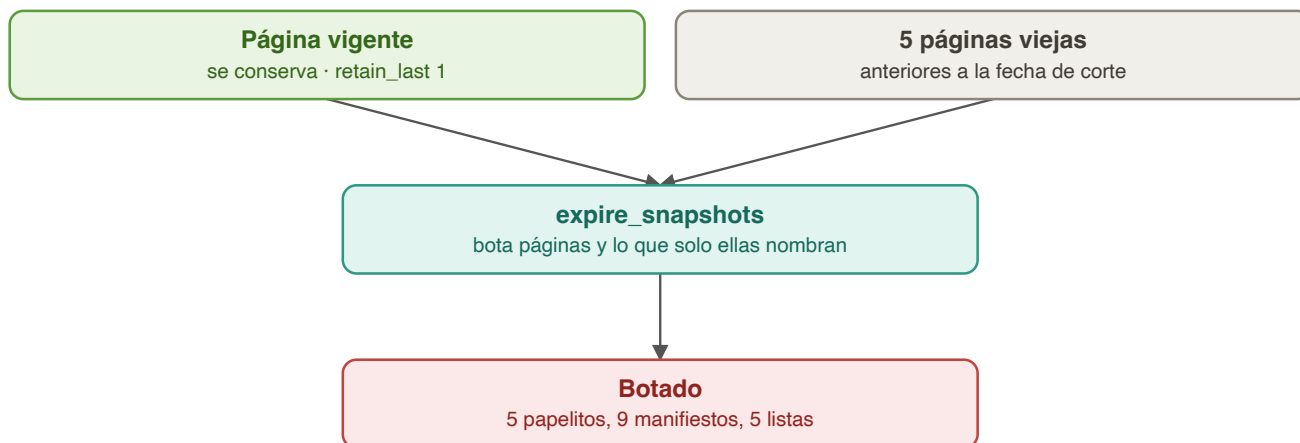
Paso 4 · Botar las páginas viejas

Celda 4.1 · Expirar las páginas

La pregunta. ¿Cómo se libera el espacio de la historia?

Por qué ahora. Ya sabes dónde se fue el espacio. Ahora lo recuperas, y eso no se deshace. Después de esta celda, pedir la tabla como estaba en una página vieja deja de funcionar.

En el almacén. Es abrir el cajón y botar las páginas viejas con los papelitos que solo ellas nombraban. Lo que se bota no se recupera.



Página vigente (verde). Se conserva siempre, porque `retain_last` pide conservar al menos una página.

5 páginas viejas (gris). Todas son anteriores a la fecha de corte que recibe el procedimiento.

expire_snapshots (verde agua). Bota las páginas viejas y, con ellas, los archivos que ninguna página que queda sigue nombrando.

Botado (rojo). El recibo de lo que se fue, que es la salida de la celda.

La sentencia, parte por parte.

- `CALL spark_catalog.system.expire_snapshots(...)` llama a un procedimiento de Iceberg. `spark_catalog` es el catálogo de Spark y `system` el grupo de procedimientos de Iceberg.
- `table => 'mi_espacio.documentos_lab11'` es la tabla, con su espacio, entre comillas. La flecha `=>` asigna un valor a cada argumento por su nombre.
- `older_than => TIMESTAMP '2030-01-01 00:00:00'` es la fecha de corte. Se botan las páginas anteriores a ese instante. La fecha es de laboratorio, en el futuro, para que toda la historia cuente como vieja y se vea el efecto completo en una sola celda. En producción, una fecha futura se lleva toda la historia de la tabla.
- `retain_last => 1` conserva siempre al menos la última página, pase lo que pase con la fecha.

```
sql
%%sql
-- Celda 4.1
CALL spark_catalog.system.expire_snapshots(
  table      => 'mi_espacio.documentos_lab11',
  older_than => TIMESTAMP '2030-01-01 00:00:00',
  retain_last => 1
)
```

Lo que sale en pantalla.

```
text
deleted_data_files_count  deleted_position_delete_files_count  deleted_equality_delete_files_count  deleted_manifest_files_count  deleted_manifest_lists_count  deleted_statistics_files_count
5                          0                          0                          9                          5                          0
1 fila.
```

Cómo se lee. Se botaron cinco papelitos, los de las cinco páginas viejas, más los manifiestos y las listas de manifiestos que solo esas páginas usaban.

- `deleted_data_files_count` es 5, los papelitos Parquet borrados.
- `deleted_position_delete_files_count` y `deleted_equality_delete_files_count` son 0. Son archivos de borrado que esta tabla no usa, porque corrige copiando papelitos enteros.
- `deleted_manifest_files_count` es 9, los manifiestos borrados. Cada corrección escribió manifiestos nuevos, y los que solo nombraban páginas viejas se fueron.
- `deleted_manifest_lists_count` es 5, una lista de manifiestos por cada página vieja.
- `deleted_statistics_files_count` es 0. Son archivos de estadísticas que esta tabla no tiene.

Celda 4.2 · Medir el disco por tercera vez

La pregunta. ¿Cuánto ocupa ahora la carpeta?

Por qué ahora. Es la última de las tres mediciones, la que dice si botar la historia sirvió. La celda viene escrita en el cuaderno.

En el almacén. Es pesar el cajón después de botar las páginas viejas.

La sentencia, parte por parte.

- `!` manda la línea al sistema operativo.
- `hdfs dfs -du -s` pide el uso de disco total de una carpeta.
- `/warehouse/iceberg/mi_espacio.db/documentos_lab11` es la carpeta de la tabla.

```
bash
# Celda 4.2
!hdfs dfs -du -s /warehouse/iceberg/mi_espacio.db/documentos_lab11
```

Lo que sale en pantalla.

```
text
842455  2527365  /warehouse/iceberg/mi_espacio.db/documentos_lab11
```

Cómo se lee. Bajó de 4,81 MB a 0,84 MB (842.455 bytes). Volvió casi al tamaño del principio.

Queda un poco más que en la primera medición por dos razones. El papelito vigente pesa algo más que el original, y en la carpeta `metadata` quedan las portadas, los archivos `metadata.json`, que se escriben uno por cada cambio y que `expire_snapshots` no borra. Son chicos.

Celda 4.3 · Comprobar los datos

La pregunta. ¿Se tocaron los datos de hoy?

Por qué ahora. Se botó la historia. Hay que comprobar que la tabla vigente quedó igual.

En el almacén. Es contar los renglones de la página de hoy, después de vaciar el cajón.

La sentencia, parte por parte.

- `count(*)` cuenta todas las filas.
- `AS documentos` le pone nombre a la columna.
- `FROM documentos_lab11` es la tabla de hoy.

```
sql
%%sql
-- Celda 4.3
SELECT count(*) AS documentos FROM documentos_lab11
```

Lo que sale en pantalla.

```
text
documentos
30000
1 fila.
```

Cómo se lee. Los mismos 30000 documentos. Se botó la historia, no los datos.

Paso 5 · Los archivos que nadie reclama

Celda 5.1 · Buscar huérfanos

La pregunta. ¿Hay archivos en la carpeta que ninguna página nombra?

Por qué ahora. `expire_snapshots` borra lo que la libreta declara como viejo. Pero pueden quedar archivos que la libreta no declara en absoluto, de escrituras que se cortaron a la mitad. Son los huérfanos, y los busca otro procedimiento.

En el almacén. Es revisar el cajón buscando papelitos que no aparecen en ninguna página de la libreta, y hacer una lista antes de botar nada.

La sentencia, parte por parte.

- `CALL spark_catalog.system.remove_orphan_files(...)` llama al procedimiento que busca y borra archivos huérfanos.
- `table => 'mi_espacio.documentos_lab11'` es la tabla.
- `older_than => TIMESTAMP '2026-09-01 00:00:00'` solo considera archivos anteriores a esa fecha. Protege a los archivos recién escritos, que pueden ser de una escritura que todavía no hace su commit.
- `dry_run => true` hace una prueba en seco. No borra nada y solo lista lo que borraría.

```
sql
%%sql
-- Celda 5.1
CALL spark_catalog.system.remove_orphan_files(
  table      => 'mi_espacio.documentos_lab11',
  older_than => TIMESTAMP '2026-09-01 00:00:00',
  dry_run    => true
)
```

Lo que sale en pantalla.

```
text
orphan_file_location
0 filas.
```

Cómo se lee. Ninguna fila. No hay huérfanos, porque ninguna escritura se cortó. `orphan_file_location` es el nombre de la columna donde saldría la ruta de cada huérfano.

Preguntas frecuentes

¿Qué son los papelitos y qué es la libreta dentro de la carpeta de la tabla?

Una tabla Iceberg es una carpeta con dos tipos de archivos.

- En `data/` están los **papelitos**, los archivos Parquet, que tienen las filas. En esta tabla, al principio, un solo papelito con los treinta mil documentos.
- En `metadata/` está la **libreta**, que no tiene ni una fila de datos. Ahí están la **portada**, un archivo `metadata.json` que dice cuál es la página vigente, la **lista de manifiestos** de cada página, un archivo `snap-...avro`, y los **manifiestos**, archivos `.avro` con la lista de papelitos de la página, su tamaño y su **rótulo** con el mínimo y el máximo de cada columna.

La celda 1.1 suma solo los papelitos de la página vigente. El disco, en la celda 1.2, suma las dos carpetas.

¿Por qué el segundo número de `hdfs dfs -du` es el triple?

Porque HDFS guarda cada archivo en tres copias, en servidores distintos, para no perderlo si uno falla. Eso es el factor de replicación, y en este ambiente vale 3. El primer número es lo que pesan los archivos, y el segundo lo que ocuparían sus tres copias.

En este ambiente hay un solo servidor de datos, así que en la práctica existe una sola copia. HDFS igual informa el espacio de las tres que pide. Por eso el número que importa para entender la tabla es el primero.

¿Cuánto es cada medición en megas?

Los megas de esta guía son de un millón de bytes.

Medición	Bytes	MB
Libreta, papelitos vigentes al inicio	776.514	0,78
Disco, primera medición	790.816	0,79
Disco, después de cinco correcciones	4.805.132	4,81
Libreta, todos los papelitos	4.668.576	4,67
Disco, después de expirar	842.455	0,84

Si le agregas `-h` a la orden, `hdfs dfs -du -s -h`, HDFS muestra los tamaños en su propia unidad, que es de 1.048.576 bytes, y los números salen un poco más chicos que en esta tabla.

¿Por qué cada UPDATE copia los treinta mil documentos?

Porque la tabla tiene un solo papelito y Iceberg no modifica archivos que ya existen. Para cambiar unas filas, reescribe el papelito completo donde están. Si la tabla tuviera muchos papelitos, por ejemplo uno por mes, cada corrección reescribiría solo los papelitos donde están las filas corregidas.

¿Puedo recuperar una página después de expirla?

No. `expire_snapshots` borra los archivos del disco. Por eso la fecha de corte se decide antes, no después.

¿Por qué la fecha de la celda 4.1 es de 2030?

Para que en una sola celda toda la historia cuente como vieja y se vea el efecto completo. En producción la fecha de corte sale de la política de retención, y una fecha futura se llevaría toda la historia salvo la página vigente, que `expire_snapshots` nunca borra.

¿Por qué no hubo huérfanos?

Porque todas las escrituras terminaron bien. Un huérfano aparece cuando un proceso escribe un papelito y se cae antes del commit. Además, la fecha de corte de la celda 5.1 deja fuera los archivos más nuevos que el 1 de septiembre de 2026, así que un archivo escrito hoy no aparecería aunque estuviera huérfano. Esa protección es a propósito, porque un archivo recién escrito puede ser de una carga que todavía no termina.

¿Cómo se decide cuánta historia guardar?

Con dos preguntas. Cuánto hacia atrás tiene que poder preguntar la auditoría, que fija el mínimo, y cuánto disco hay, que fija el máximo. Entre esos dos se elige la fecha de corte, y la rutina de `expire_snapshots` se programa con esa regla.

Cierre

La pregunta. ¿Qué te llevas de este laboratorio?

La libreta guarda todo hasta que alguien le dice qué botar. Eso no es un defecto, es lo que permite viajar al pasado. Pero cuesta disco, y cuánta historia se conserva es una decisión que hay que tomar.

En el almacén. El cajón se llenó con copias de la misma libreta, una por corrección. Nadie las botó hasta que el dueño decidió, y cuando las botó el cajón volvió a cerrar. Las anotaciones de hoy no se tocaron.

Las tres mediciones del disco.

Momento	Disco, en bytes	MB	Filas
Después de cargar	790.816	0,79	30.000
Después de cinco correcciones	4.805.132	4,81	30.000
Después de expirar	842.455	0,84	30.000

Si nadie toma la decisión de retención, el disco crece con cada corrección, aunque las filas sean siempre las mismas.